Chapter 5
An Adaptive Index Structure for Similarity Search

Converting a feature vector to a lower dimensional space helps in indexing the data. The trade-off is usually compromising on the effectiveness of the descriptor. For example, in Chapter 4 we observed that the retrieval accuracy of the descriptor is reduced in the lower dimensional space compared to the original feature vector. The alternative is to investigate methods that can facilitate efficient search in the high dimensional feature space directly. This high dimensional search is the focus of this chapter. We propose a new adaptive index structure for similarity search that appears quite promising for image retrieval applications.

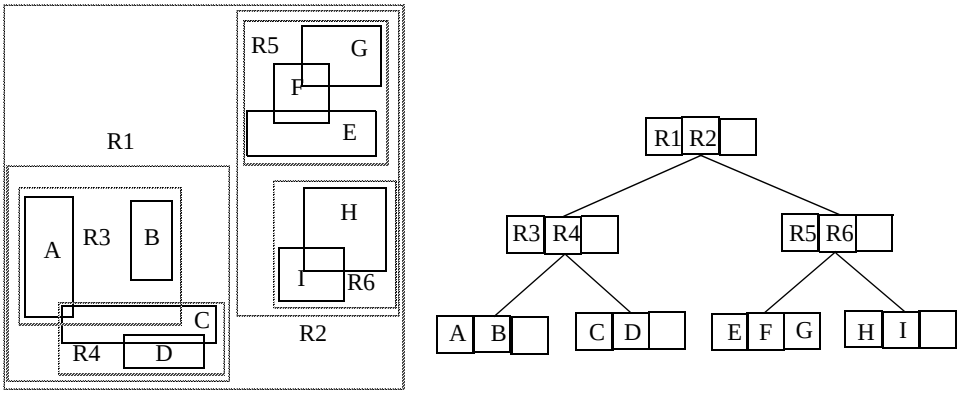
5.1 Introduction

To index high dimensional feature vectors, various index structures have been proposed [10, 11, 16, 26, 30, 31, 44, 66, 83, 84]. One can broadly classify them into data partitioning (DP) and space partitioning (SP) techniques, depending on how data are grouped. A data partitioning based index structure consists of bounding regions arranged in a containment hierarchy. At the data level, the nearby data items are clustered within bounding regions. At a higher levels, nearby bounding regions are recursively clustered within bigger bounding regions, thus forming a hierarchical directory structure. The bounding regions may overlap with each other. The bounding regions can be bounding boxes or bounding spheres/diamonds (e.g. SS-tree [84], TV-tree [44]). Figure 5-1(a) illustrates a data partitioning based index

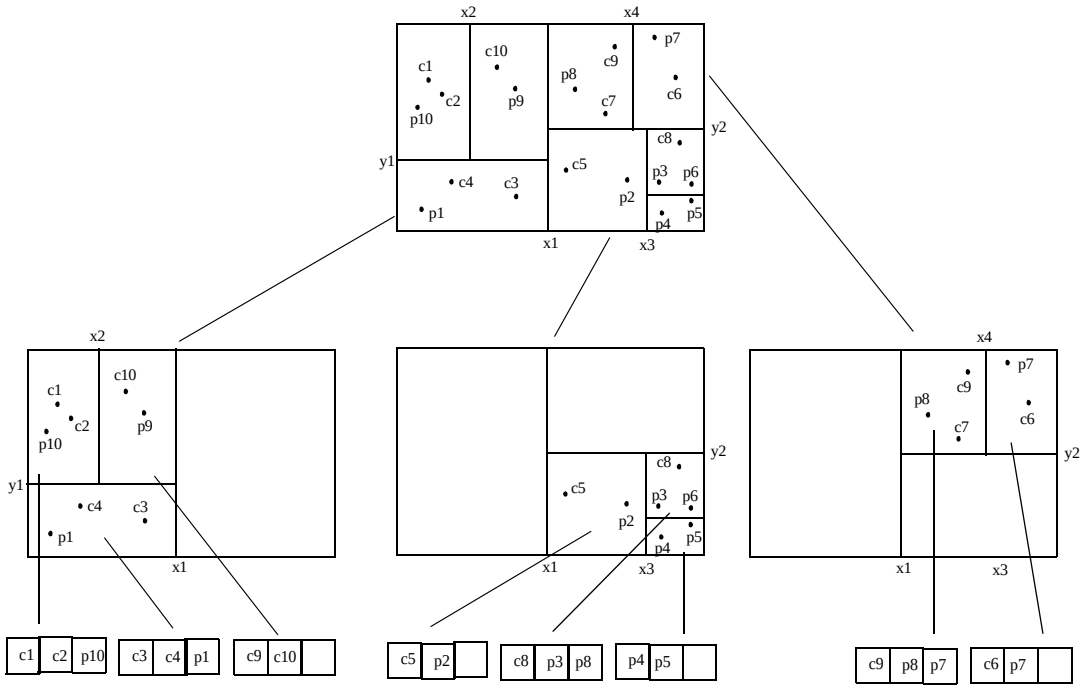
structure, R-tree [31]. It uses the bounding boxes to group objects and the regions at each layer can overlap, see region R4 and R3 for example. On the other hand, a space partitioning based index structure consists of space recursively petitioned into mutually disjoint subspaces. The hierarchy of partitions form the tree structure. Such an index structure, k-d-B tree [66], is illustrated in Figure 5-1(b). As observed, the union of the regions at the same tree level is the complete data space.

Alternatively, they can be classified into feature based (e.g. R-tree [31], k-d-B tree [66]) and distance based (e.g. SS-tree [84], TV-tree [44]) methods. In feature based techniques, the data/space partitioning is based on th values of the vectors along each independent dimension and is independent of the distance function used to compute the distance among objects in the database or between query objects and databases objects. For example, for the k-d-B tree as illustrated in Figure 5-1(b), the split of each node is related with the value of the inserted data items along a certain dimension and the number of items after splitting (to ensure a balanced tree), but independent from the distances among items. Consider splitting the root node of k-d-B tree in Figure 5-1(b), having x1 determined as one discriminator, whether an inserted item will be assigned to the most left node at the second level of the tree is determined by if its value on x-axis is smaller than x1, but has nothing to do with the distance of the inserted item with other items. On the other hand, distance based techniques partition data space based on the distance of objects from one or more selected pivot point(s), where the distance is computed using a given distance function. An example of distance-based index structure is SS-tree, in which the bounding sphere is used to group the items. The minimum bounding sphere is characterized by a centroid and a radius, an item will be assigned to one of the bounding spheres if it has the minimum distance to that sphere. The distance needs to be fixed before the construction of the index structure.

The index structures introduced above generally work well for low-dimensional space, such as the feature space consisting of data from geosciences, mechanical CAD related applications. But their performance is known to degrade as the number of dimensions increases. Though some index structures, including X-tree [10], R*-tree [8], TV-tree [44], SS-tree [84], are particularly designed to access high dimensional data space, there is no generally recog-



(a) R-tree



(b) k-d-B tree

FIGURE 5-1. Examples of tree based index structures. (a) R-tree; (b) k-d-B tree

nized technique that can overcome this degradation caused by the high dimensionality. A recent research [83] provides a quantitative analysis of most well-known tree-based index structures in performing nearest neighbor search. It argues that typical tree-based index methods are outperformed by a linear search when the search dimensions exceed a moderate value, 10 for example. The analysis of [83] assumes the uniform distribution of the feature space and the independence of dimensions. For space partitioning based index structures, it concludes that statistically, to find a nearest neighbor, the probability for any subspace to be accessed is 1 when the dimensionality is high enough. In another words, all the subspaces have to be visited. For data partitioning based index structures, the analysis demonstrates that due to the overlap of the minimum bounding regions, the percentage of bounding regions visited increases rapidly, and reaches 100% when the dimensionality is beyond a threshold. Though this work is an average case study and the assumptions made to facilitate the analysis may be questionable, it provides a good reference to understand the challenge of indexing high dimensional feature space.

Besides the high dimensionality of the feature space, the research in utilizing the image/video information suggests there are other factors should be considered in the design of index structures

- similarity metric: Traditionally, it is assumed that the distance metrics are fixed at the design time. However, for image/video objects, there are no well defined similarity metrics that work in all cases. In most information retrieval systems that involve the similarity comparison of image/video objects, the Minkowski distance is adopted, which is of the form as following,

$$d(A, B) = \left[\sum_i (A_i - B_i)^p\right]^{\frac{1}{p}} \tag{5-1}$$

where A and B are the feature representation of any two objects and $A = \{A_1, \ldots, A_N\}$, $B = \{B_1, \ldots, B_N\}$. $p > 0$ is a constant which characterizes the distance function, for Manhattan distance $p = 1$, for Euclidean distance $p = 2$. But the psychological literature to model human similarity perception also suggests other alternatives [71]. Some proposed similarity measures may not even obey the distance

axioms, such as the fuzzy set-theoretic measure proposed in [71]. To compensate the deficiency of using a rigid similarity measure, one popular solution is to modify the similarity measure as defined in (5-1) to be a weighted one and allow the weights to be determined on the fly to adapt the similarity measure to a specific application or a subspace of the whole feature space. Some recent works have been presented for efficiently adjusting the weights [38, 55, 65, 70]. The need to adjust the similarity measure essentially demands to separate the similarity measure from the design of the index structure. In another words, the feature based techniques [10, 11, 26, 66, 83] are more preferred than the distance based techniques [44, 84]. For example, if the SS-tree [84] is used as the index structure to organize the image objects according to a pre-assigned distance measure. The changing of the distance measure will result in different centroid and radius of a bounding sphere, which in turn requires changing the physical storage of the feature data. For a large database, such change is not acceptable because of the high cost. What can be worse is that a set of image/video objects is typically accessed by multiple applications. If different applications require different similarity measures, there have to be multiple copies of the feature data organized according to each of the similarity measures. But considering to use the k-d-B tree as illustrated in Figure 5-1(b) to organize the feature space, the division of the feature space is based on the values on each dimension independently. As the values of feature vectors are invariable for different similarity measures, so is the constructed index structure. Of course, for a database oriented for a fixed similarity measure, distance based approach can be better tuned.

- distribution of the feature vectors: the index structure should be adaptive to the data distribution. It is often assumed that the feature space is uniformly populated. This is far from reality for many audio-visual descriptors. For example, for the Brodatz texture image data [14], the corresponding descriptors tend to cluster in the 60-dimensional texture descriptor space. Many existing index structures that make the assumption that the data is uniformly distributed do not perform well on such non-uniformly distributed data.

Motivated by the analysis of the impact of the high dimensionality on tree-based index structures, the authors of [83] introduce a index structure, called Vector Approximation File (VA-File) for the nearest neighbor search in high dimensional space. The focus of the VA-File is how to perform the linear scan in a more efficient fashion, rather than how hierarchically grouping the data space. The main idea of VA-File is to first construct a compressed domain representation of the feature space, in which the value of a feature vector can be approximately represented. The compression of the feature space enables more items can be loaded into the main memory for fast access. Then when the nearest neighbor search is performed, it will go through the compressed domain to exclude large amount of data items that can not be the nearest neighbor of a query by examining the approximations. Only for a small amount of items, their feature vectors need to be accessed to find the nearest neighbor. The evaluation presented in [83] demonstrates that VA-File outperforms some well-known tree based index structures (X-tree, R*-tree) in high dimensions.

The use of a compressed representation of feature space to speed up the search can also be found in a recent work [11]. In [11], a multi-layer index structure, called IQ-tree, is proposed. The first layer of IQ-tree uses the bounding box to perform data partitioning. For data within each bounded box, the approximation of feature vectors is constructed similar with [83] to support intra-box searching. The main research issue of this work is how to adaptively combine the ability of using tree structure to index low dimensional space and the robust performance of using compressed representation to search in high dimensions into a unified index structure.

For image/video databases, approximation based methods have certain advantages. First, the construction of the approximation is based on the feature values on each of the dimensions independently, similar with k-d-B tree. As a result, the compressed domain representation can support different distance measures. This is a very important property especially for learning and concept mining related applications. Secondly, the independence of dimensions brings the advantage of parallelism in constructing the index structure. Considering the large dimensionality of image/video descriptors, this property is very attractive from implementa-

tion point of view. However, there are also some concerns that need to be addressed in effectively using such approximation based method.

Approximation based methods are sensitive to data distribution. The approximations in [11] and [83] are constructed by using equally spaced grids on each dimension. As a result, the assumption that data is uniformly distributed plays an important role in effectively indexing objects. As mentioned earlier, this assumption is not valid for many image/video feature descriptors. For this reason, an approach is proposed in [26] to adapt the construction of the approximation to statistical properties of data. It first finds a set of axes using KL-transform and maps the original feature space into the transformed space in which the data is less correlated. In the mapped space, the data in each dimension is clustered into a pre-assigned number of grids using the Lloyd's algorithm [29]. The approximation constructed in this way partitions the feature space into hyper rectangles rather than hyper cubes. It is reported in [39] that for high dimensional data, projecting the data onto a selected set of axes does not result in a significant decorrelation of the data. This implies that global statistics, such as the second order statistics used in [26], may not be able to characterize the data distribution effectively for high dimensional spaces. Instead, our proposed method is based on characterizing the data using the marginal distributions which are easy to compute and offer significant overall performance improvement. In the following, we refer the uniform partitioning adopted in [11, 83] as "uniform approximation".

In this work, we propose an approach to adapt the construction of the approximation to data's marginal distribution as a way of improving the index efficiency. The main idea is to first estimate the marginal distribution of the data. Using the marginal distribution information, each of the data dimensions is then partitioned into bins such that each bin contains approximately an equal number of objects. The main objective is to create bins in the high dimensional space that are approximately equally populated, but with much less computational cost.

Data's marginal distribution is characterized by its marginal probability density function (pdf). The estimation of pdf is independent of the distance measure. The data's distribution

can also be studied by clustering methods [26, 88]. Clustering requires specifying a distance measure, which is not desirable.

The estimation of the pdf for each dimension can be done independently. We propose using mixture of Gaussians to model data's marginal distribution. Since we have no any prior knowledge about the possible distribution, using mixture of Gaussians provides the flexibility to model any possible distribution with good approximation [23]. The expectation-maximization (EM) algorithm [23] is used to estimate the parameters of the model.

In Section 5.2, we first briefly summarize the construction of uniform approximation and its associated indexing. We also discuss the limitations of using uniform approximation. Our proposed method based on marginal distributions is presented in Section 5.3. Experimental results are provided in Section 5.4. Section 5.5 concludes with a summarization.

5.2 Indexing high dimensional data using uniform approximation

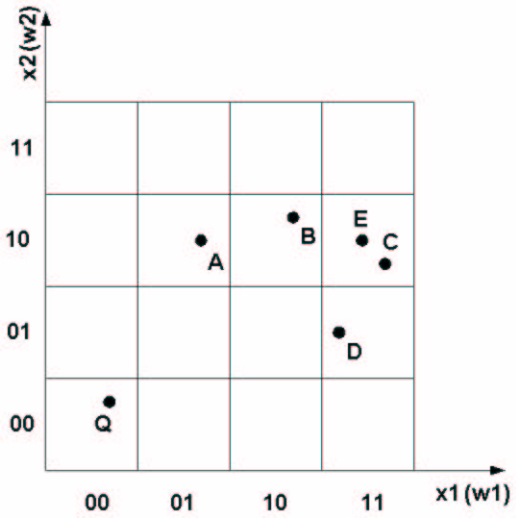
5.2.1 Construction of uniform approximation [83]

In this section we summarize the VA-File data structure presented in [83]. This data structure is based on uniformly quantize each of the data dimensions. Consider a database of N, M-dimensional feature vectors $F_i = [f_{i1}, f_{i2}, \ldots, f_{iM}]$, $i = 1, \ldots, N$. For example, these feature vectors may correspond to the color histograms of images in the database. Let $Q = [q_1, q_2, \ldots, q_M]$ be the query object. A popular dis-similarity metric - the weighted Euclidian distance - is used for retrieving the nearest neighbors. The distance between a database object F_i and the query Q is computed as:

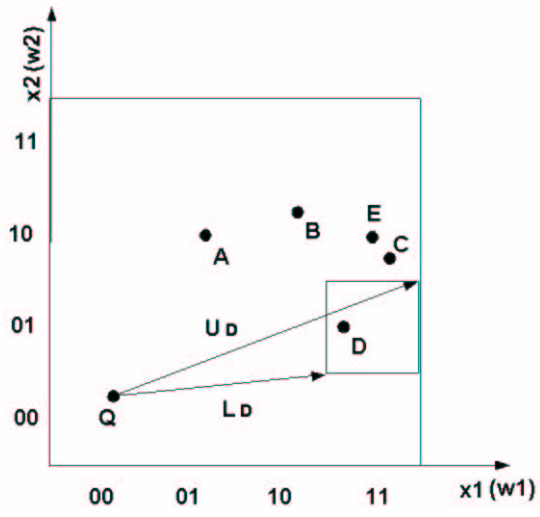
$$d(Q, F_i, W) = (Q - F_i)W(Q - F_i)^T \tag{5-2}$$

where W is a real symmetric matrix [9, 14, 15, 16]. Given such a M dimensional feature space, the corresponding approximation based index structure is constructed as follows.

First, each of the feature vector dimensions is partitioned into a certain number of non overlapping segments. Typically, the number of segments is assigned to be 2^{B_j},



(a)



(b)

FIGURE 5-2. (a) Construction of the approximations; (b) Computation of the lower bound and the upper bound.

$j = 1, \ldots, M$, where B_j is the number of bits allocated to dimension j. Denote the boundary points that determine the 2^{B_j} partitions to be $b_{l,j}$ $l = 0, 1, \ldots, 2^{B_j} - 1$. A given partition

$G^j(l)$ is determined by $[b_{l,j}, b_{l+1,j}]$. Thus, each high dimensional cell is represented by a string of bits of length B

$$B = \sum_{j = 1, \ldots, M} B_j \tag{5-3}$$

For a feature vector F_i, its approximation $P(F_i)$ is an index to the cell containing F_i. Consider Figure 5-2(a), where $B_1 = B_2 = 2$. The approximation for $P(A)$ is "0110", where "01" and "10" are the indices on dimension x_1 and x_2 respectively. Note that each approximation cell contains a number of feature vectors. For example, in Figure 5-2(a), $P(C)$ and $P(E)$ have the same approximation "1110".

Note that if F_i is in partition l along the jth dimension, then

$$b_{l,j} \le f_{ij} \le b_{l+1,j} \tag{5-4}$$

Given a query feature vector Q and a weight matrix W_t, it is easy to verify that [83]

$$L_i(Q, W) \le d(Q, F_i, W) \le U_i(Q, W) \tag{5-5}$$

where the lower bound $L_i(Q, W)$ and upper bound $U_i(Q, W)$ are computed as

$$L_i(Q, W) = [l_{i1} \;\; l_{i2} \;\; \ldots l_{iM}] W [l_{i1} \;\; l_{i2} \;\; \ldots l_{iM}]^T \tag{5-6}$$

$$U_i(Q, W) = [u_{i1} \;\; u_{i2} \;\; \ldots u_{iM}] W [u_{i1} \;\; u_{i2} \;\; \ldots u_{iM}]^T \tag{5-7}$$

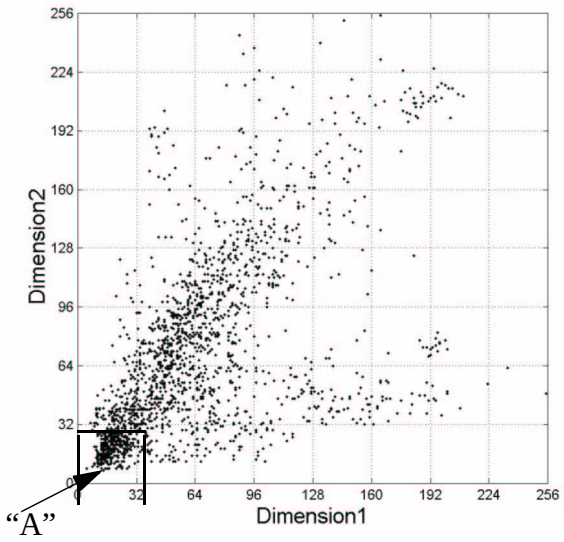
where

$$l_{ij} = \begin{cases} q_j - b_{l+1,j} & q_j > b_{l+1,j} \\ 0 & q_j \in [b_{l,j}, b_{l+1,j}] \\ b_{l,j} - q_j & q_j < b_{l,j} \end{cases}$$

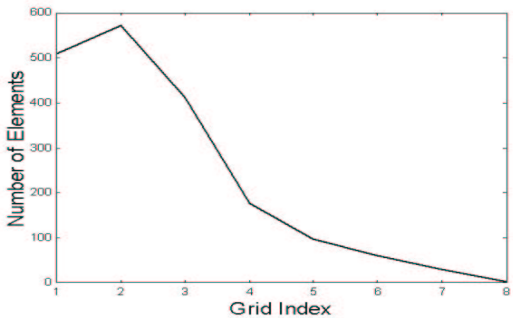
$$u_{ij} = \begin{cases} q_j - b_{l,j} & q_j > b_{l+1,j} \\ max(q_j - b_{l,j}, b_{l+1,j} - q_j) & q_j \in [b_{l,j}, b_{l+1,j}] \\ b_{l+1,j} - q_j & q_j < b_{l,j} \end{cases} \tag{5-8}$$

Figure 5-2(b) illustrates these bounds for the 2-D examples in Figure 5-2(a).

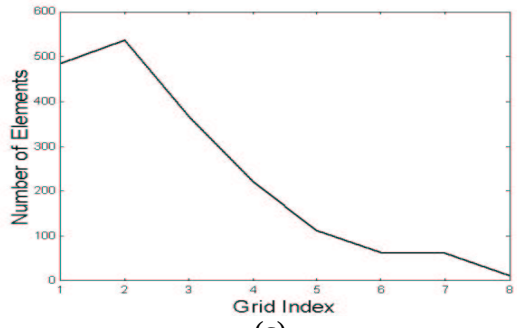
Figure 5-3(a) gives an illustrative example of constructing the regular approximation for two dimension data ($N = 1856$, $B_1 = B_2 = 3$). The 1856 image objects are collected from the Brodatz album [4,12]. Images in the Brodatz album are usually considered to be represen-



(a)



(b)



(c)

FIGURE 5-3. (a) Using regular approximation to uniformly partition a two dimensional space. $N = 1856$, $B_1 = B_2 = 3$; (b) The histogram of data in dimension 1; (c) The histogram of data in dimension 2. Note that in (b) and (c), the grid index indicates the partitions in (a).

tatives of real world textures. Figure 5-3(a) shows the first two components of the 60 dimensional feature vectors computed as in [52]. These two dimensions are uniformly partitioned, with 3 bits assigned to each dimension, resulting in a total of 64 cells (Figure 5-3(a)). Figure 5-3(b) and Figure 5-3(c) show the distributions of the data along these two dimensions. As can be observed from Figure 5-3(b) and (c), the feature distribution is not so uniform. Similar observations have also been made in [26].

5.2.2 Indexing based on uniform approximation

Approximation based nearest neighbor search can be considered as a two phase filtering process. In the first phase, the set of all vector approximations is scanned sequentially and lower and upper bounds on the distances of each object in the database to the query object are computed. Obviously, the actual distance between the query object and a database object is smaller (larger) than the corresponding upper (lower) bound. In this phase, if an approximation is encountered such that its lower bound is larger than the k-th smallest upper bound found so far, the corresponding feature vector can be skipped since at least k better candidates exist. At the end of the first phase filtering, the set of vectors that are not skipped are collected as candidates for the second phase filtering. Denote the number of elements in the candidate set to be N_1.

In the second phase filtering, the actual N_1 feature vectors are examined. The feature vectors are visited in increasing order of their lower bounds and then exact distances to the query vector are computed. If a lower bound is reached that is larger than the k-th actual nearest neighbor distance encountered so far, there is no need to visit the remaining candidates. Let N_2 denote the number of feature vectors visited before the thresholding lower bound is encountered.

For approximation based methods [26, 83], the index performance is usually measured by the values of N_1 and N_2. N_1 measures the block selectivity and N_2 is used to measure the I/O cost (see [83]). Smaller values of N_1 and N_2 indicate a better performance.

5.2.3 Limitations of uniform approximation based indexing

The effectiveness of using VA-File based indexing structure relies on how uniform the data's distribution is. For the example given in Figure 5-3, the data on each dimension is not uniformly distributed, as observed from the histograms shown in Figure 5-3(b) and Figure 5-3(c). As a result, if a query vector happens to be one that falls into the cell "A" in Figure 5-3(a), in which approximately 30% of elements are contained, we will have $N_1 > 0.3N$ and $N_2 > 0.3N$, which indicates a poor selectivity in the first phase filtering and a high I/O cost for the second phase filtering.

This simple example illustrates one of the shortcomings of the uniform approximation, when the data distribution is not uniform. In such cases, the first phase of filtering still results in a large number of items to search for during the more expensive second phase computations. Our proposed method overcomes this problem by adapting to the marginal data distribution, thus resulting in a significant performance enhancement over the uniform approximation methods.

5.3 Approximation based on marginal distribution

In the above discussion, we have noted that densely populated cells can potentially degrade the indexing performance. A possible solution is to make the construction of the approximation adaptive to the data distribution. The general idea is to first estimate data's marginal distribution in each dimension. Secondly, individual axes are partitioned such that the data has equal probability of falling into any partition. The number of partitions are determined by the given number of bits assigned to that dimension. The approximation constructed in this way reduces the possibility of having densely populated cells, which in turn improves the indexing performance.

It should be noted that data's marginal distribution can only capture partial information of data's high dimensional distribution. For example, as shown in Figure 5-4, even if data's marginal distribution is uniform on each dimension, the joint distribution of data can still be not uniform or clustered. But considering that the correlation of data on different dimensions decreases with the increase of data's dimensionality [10], it is reasonable to expect the marginal distribution as a reasonable alternative to characterize the data distribution.

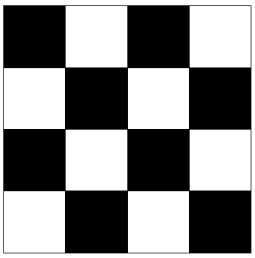


FIGURE 5-4. A non-uniform two dimensional joint pdf with uniform marginal distributions. The black blocks are zeros and the white blocks are some constant values. Clearly, the joint pdf is not uniform. But if we compute the marginal distributions by projecting the joint pdf to horizontal and vertical directions, the marginal distributions are uniform.

5.3.1 pdf modeling using mixture of Gaussians

Denote $p_i(x)$ to be the pdf of data on dimension i. The algorithm introduced below is applied to data in each dimension independently. For notation simplicity, we denote $p(x)$ to be the pdf of an one dimension signal. We model the one-dimensional pdf using a mixture of Gaussians. Specifically, we assume that $p(x)$ can be represented as

$$p(x) = \sum_{j=1}^{D} p(x|j)P(j) \tag{5-9}$$

where D is the number of Gaussians used in modeling and

$$p(x|j) = \frac{1}{\sqrt{2\pi\sigma_j^2}} \exp\left\{-\frac{(x-\mu_j)^2}{2\sigma_j^2}\right\} \tag{5-10}$$

The coefficients $P(j)$ are called the mixing parameters, which satisfies

$$\sum_{j=1}^{D} P(j) = 1 \text{ and } 0 \le P(j) \le 1 \tag{5-11}$$

In short, we model the pdf to be a weighted sum of a set of Gaussian functions. The task of estimating the pdf is then converted to be a problem of parameter estimation. The parameters we need to estimate are $\phi_j = \left\{P(j), \mu_j, \sigma_j^2\right\}$, for $j = 1, \ldots, D$.

5.3.2 Parameter estimation using the EM algorithm

Estimating parameters that determine an unknown pdf from given data is a classical statistical learning problem. One frequently used estimator is the maximum likelihood (ML) approach. The basic idea of ML approach is that the optimal estimation of parameters should result in a pdf which most likely give rise to the observed distribution of the given data. In our case, we want to find ϕ_j, $j = 1, \ldots, D$, to maximize

$$\Phi(\phi_1, \ldots, \phi_D) = \prod_{l=1}^{N} p(v[l]|(\phi_1, \ldots, \phi_D)) \tag{5-12}$$

where $v[l]$, $l = 1, \ldots, N$, are the given data set on a specific dimension.

A simple and practical method of solving this optimization problem is to use the Expectation-Maximization algorithm [6]. Given N data $v[l]$ available as the input for the estimation, EM algorithm estimates the parameters iteratively using all the N data in each iteration. Let t denote the iteration number. Then the update equation for the various parameter can be derived as (see [23])

$$\mu_j^{t+1} = \frac{\sum_{l=1}^{N} p(j|v[l])^t v[l]}{\sum_{l=1}^{N} p(j|v[l])^t} \tag{5-13}$$

$$(\sigma_j^2)^{t+1} = \frac{\sum_{l=1}^{N} p(j|v[l])^t (v[l] - \mu_j^t)^2}{\sum_{l=1}^{N} p(j|v[l])^t} \tag{5-14}$$

$$P(j)^{t+1} = \frac{1}{N} \sum_{l=1}^{N} p(j|v[l])^t \tag{5-15}$$

Using Bayes' theorem [6], the $p(j|v[l])^t$ is computed as

$$p(j|v[l])^t = \frac{p(v[l]|j)^t P(j)^t}{\sum_{i=1}^{M} p(v[l]|j)^t P(j)^t} \tag{5-16}$$

where

$$p(v[l]|j)^t = \frac{1}{\sqrt{2\pi(\sigma_j^2)^t}} \exp\left\{-\frac{(v[l]-\mu_j^t)^2}{2(\sigma_j^2)^t}\right\} \tag{5-17}$$

Figure 5-6 shows an example of a histogram (Figure 5-6(a)) and its approximation using the mixture of Gaussians (Figure 5-6(b)). As observed, data's distribution is well approximated by the estimated pdf.

One note on parameter estimation using the EM. The singular points in the distribution may cause numerical problems in computing (5-13)-(5-17). The pdf of a singular point is a Gaussian function centered at a single value. The peak of the Gaussian function approaches infinity as the variance of the function approaches zero. Such infinity can cause unpredictable results in computing (5-13)-(5-17). One approach to avoid this problem is to introduce some new data set during the iterations [79].

5.3.3 Updating parameters using on-line estimation

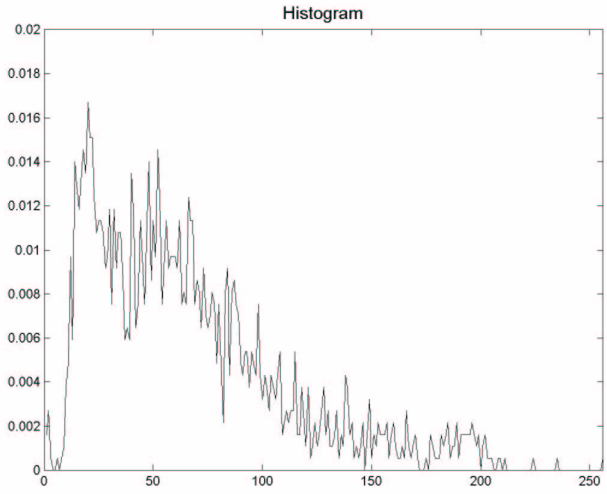
Using (5-13), (5-14) and (5-15), the parameters can be estimated given that the N data values are available. For a large database, N is usually only a small portion of the total number of elements in the database. In practice, it is desirable to have an incremental pdf update scheme that can track the changes of the data distribution. The EM algorithm can be modified for sequential updating [15]. Given that $\left\{P(j)^N, \mu_j^N, (\sigma_j^2)^N\right\}$ is the parameter set estimated form using the N data $v[l]$, the updated parameter set, when there is a new data $v[N+1]$ coming in, can be computed as

$$\mu_j^{N+1} = \mu_j^N + \theta_j^{N+1}(v[N+1]-\mu_j^N) \tag{5-18}$$

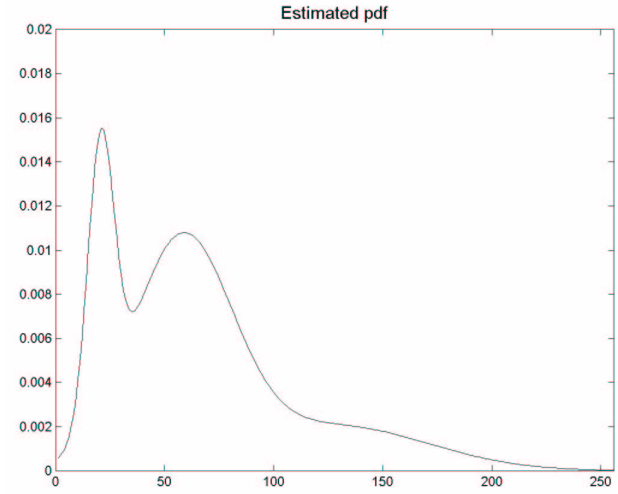
$$(\sigma_j^2)^{N+1} = (\sigma_j^2)^N + \theta_j^{N+1}[(v[N+1]-\mu_j^N)^2 - (\sigma_j^2)^N] \tag{5-19}$$

$$P(j)^{N+1} = P(j)^N + \frac{1}{N+1}(p(j|v[N+1]) - P(j)^N) \tag{5-20}$$

where



(a)



(b)

FIGURE 5-5. (a) A histogram from an aerial image texture data set; (b) The pdf estimated using the EM algorithm.

$$(\theta_j^{N+1})^{-1} = \frac{p(j|v[N])}{p(j|v[N+1])}(\theta_j^{N})^{-1} + 1 \tag{5-21}$$

The conditional probability $p(j|v[N+1])$ is computed as in (5-16) and (5-17).

Figure 5-6 illustrates this sequential updating. Figure 5-6(a) shows the synthetically generated data (5000 objects) in time sequenc.Its histogram is shown in Figure 4(b). Each individual element is added for estimation sequentially. The parameters are computed according to equation (5-18), (5-19) and (5-20). Figure 4(c), (d) and (e) show the estimated pdf when the amount of elements used in the estimation are 1700, 3400 and 5000, respectively. Clearly, as more and more data elements added sequentially, the accuracy of the estimated pdf gets better.

5.3.4 pdf quantization

Denote the estimated pdf to be $p(x)$ as the approximation of $p(x)$. The objective of nonlinear quantization is to segment the pdf into grids of equal area. If the boundary points are denoted by $c[l]$, $l = 0, \ldots, 2^b$, b is the number of bits allocated, the boundary points should satisfy

$$\int_{c[l]}^{c[l+1]} \hat{p}(x)dx = \frac{1}{2^b}\int_{c[0]}^{c[2^b]} \hat{p}(x)dx \tag{5-22}$$

Using this criterion, the boundary points can be determined from a single scan of the estimated pdf. In [7], the boundary points of each dimension are determined by clustering all the N data into 2^b clusters, which is computationally expensive. In contrast, using () to determine the boundary points is quite inexpensive, and does not require a distance measure.

5.3.5 Approximation updating

For dynamic databases, the pdf estimates are to be updated periodically. In our implementation, we update the estimates whenever a certain number of new data items are added. The approximation and pdf quantization are updated only when the new pdf, denoted as $p_{new}(x)$, differs significantly from the current pdf, denoted as $p_{old}(x)$. We use the following measure to quantify this change

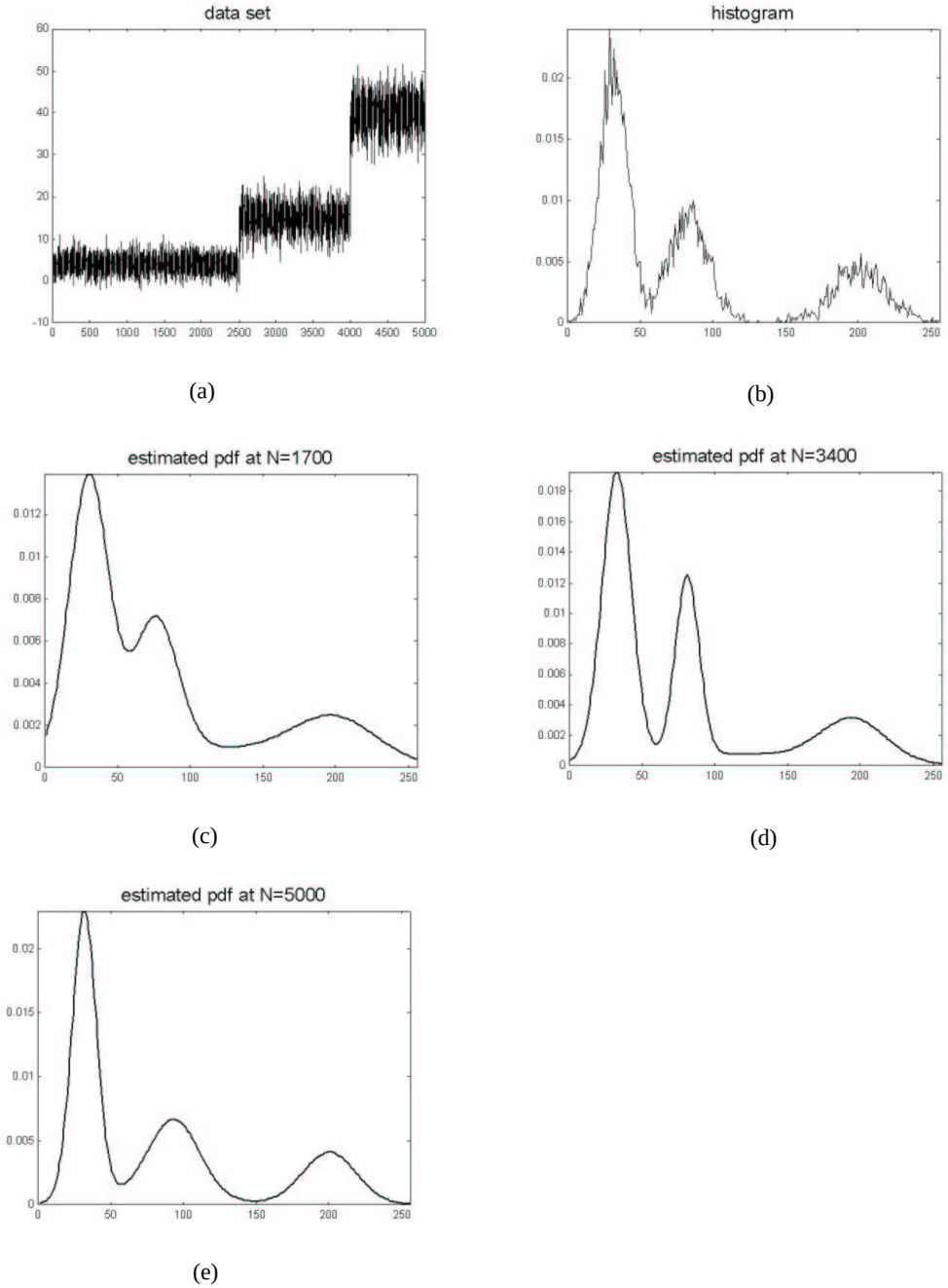


FIGURE 5-6. On-line estimation: (a) original data set N=5000; (b) histogram of the data set; (c) estimated pdf when 1700 elements are used; (d) estimated pdf when 3400 elements are used; (e) estimated pdf when 5000 are used

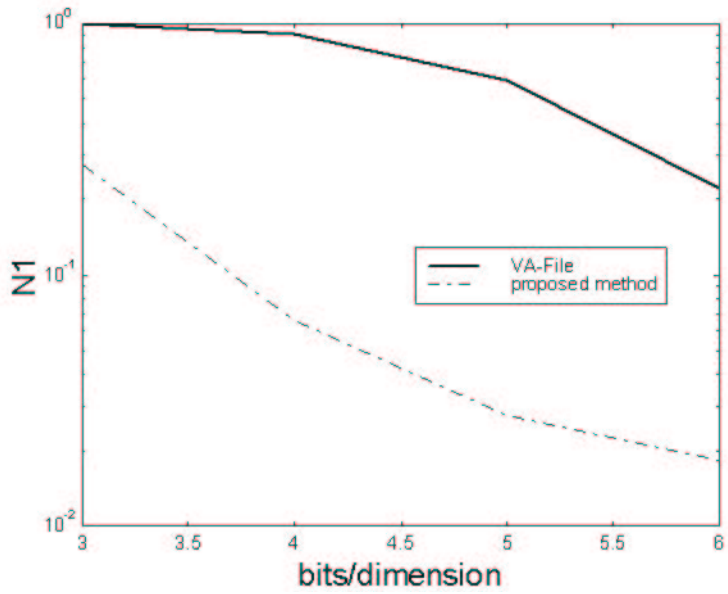
$$\rho = \frac{\int (\hat{p}_{old}(x) - \hat{p}_{new}(x))^2 dx}{\int \hat{p}_{old}(x)^2 dx} \quad (5\text{-}23)$$

The approximation on that dimension is updated when ρ is larger than a certain threshold.

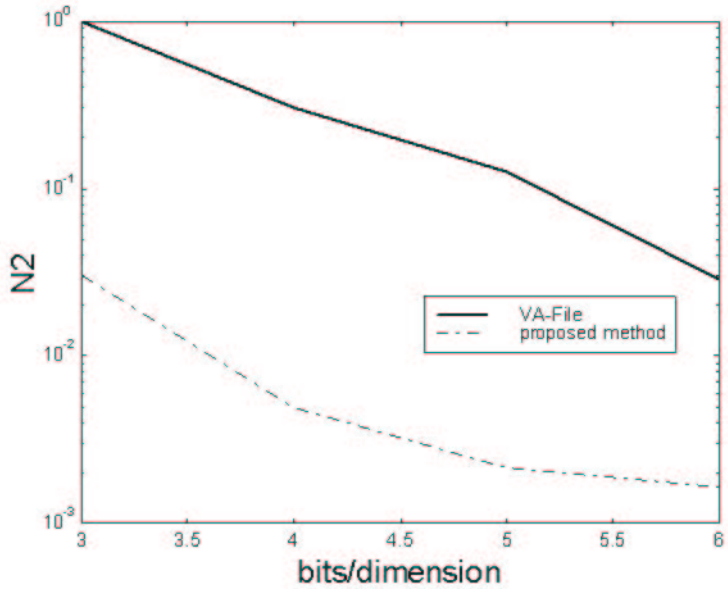
5.4 Performance evaluation

Our preliminary evaluation is performed on a database containing 275,465 aerial photo images. For each image, the texture descriptor HTD as introduced in Chapter 2 is adopted to compute a 60 dimensional feature vector to characterize image's texture feature. Using the algorithms proposed in Section 5.3, we developed an adaptive indexing scheme to demonstrate its usage for realistic applications. Initially, around 10% of total number of image objects are used to initialize the pdf estimation using the algorithms proposed in Section 5.3.2. Based on the estimated pdf, the approximation is constructed to support nearest neighbor search of the whole database. Meanwhile, an updating strategy is developed to take the rest of the data as input to the on-line estimation. For each dimension, when the change of the estimated pdf, which is measured by (), is beyond the threshold $\rho > 0.15$, the approximation is adjusted accordingly.

The approach that uses the regular approximation, VA-File, is also implemented for comparison purpose. The number of candidates from the first phase filtering N_1 and the number of visited feature vectors resulting from the second phase filtering N_2 are used to evaluate the performance. We tested using 3, 4, 5 and 6 bits for each dimension to construct the approximation. For each approximation, 10 nearest neighbors search is performed on the whole database. The results are shown in Figure 5-7(a) and (b) for N_1 and N_2, respectively. Note that the figures are plotted on a logarithmic Y-axis. To normalize the results into the same range for displaying, the maximum value of $N_1(N_2)$ of VA-File is used to normalize all the values of $N_1(N_2)$. The number of candidates remaining after the first phase filtering (N_1) for the VA-File is about 3 to 20 times more than the proposed adaptive method (Figure 5-



(a)



(b)

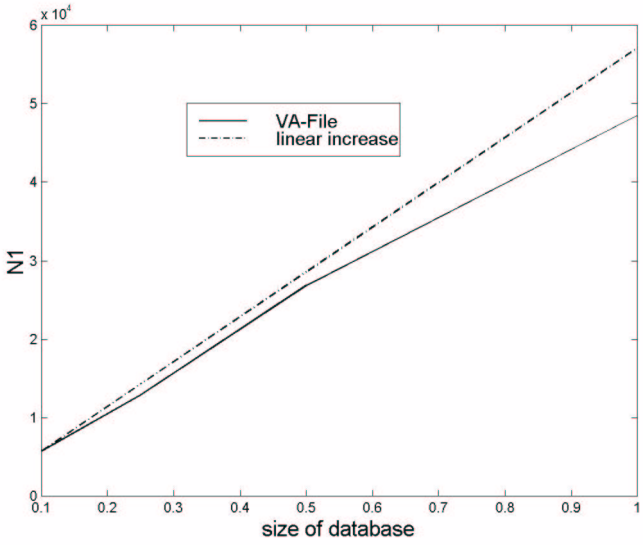
FIGURE 5-7. Comparison between VA-File and the proposed method: (a) Number of candidates (N1) remaining after the first phase filtering; (b) Number of visited feature vectors (N2) after the second phase filtering.

7(a)). After the second phase filtering, the VA-File visits 16 to 60 times more feature vectors than the proposed method (Figure 6(b). One can conclude that the adaptive pdf quantization results in an order of magnitude performance improvement over the regular VA-File.

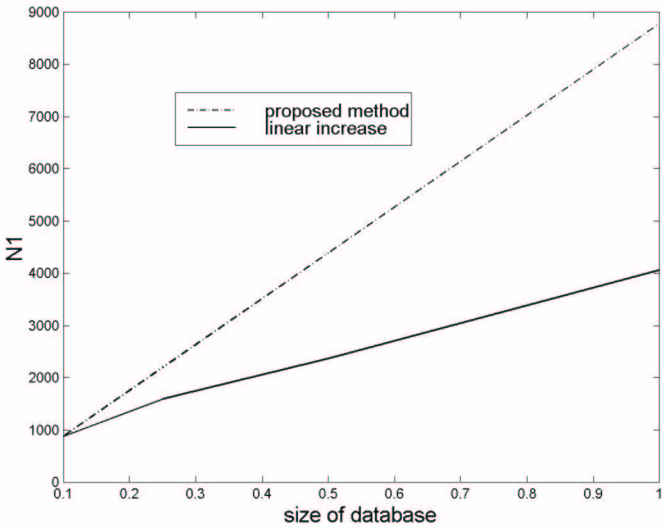
We have also investigated the indexing performance as the size of the database grows. For this purpose, we construct 4 databases, including 10%, 25%, 50% and 100% percent of the total 275,645 image objects. For each database, 6 bits are assigned to each dimension to construct the approximation. In measuring the scalability, we are mainly concerned with the number of candidates after the first phase filtering, N_1, which is directly proportional to the I/O cost of the index structure. The results for VA-File and the proposed method are shown in Figure 5-8. As can be observed from Figure 5-8, the magnitudes of N_1 resulted from VA-File are much larger than those of the proposed method. In terms of the scalability with the size of databases, the proposed method tends to maintain a better sub-linear behavior as the database grows.

5.5 Summary

We have presented a novel adaptive indexing scheme to index high dimensional image/video objects. The design of the index structure adapts to data's distribution and can supports various distance metrics. Its indexing performance scales well with the size of the database. Currently, we are evaluating the performance on a much larger database, and hope to report the details on a 1-million image data set at the conference. We will also present results on other larger dimensional datasets using MPEG-7 descriptors.



(a)



(b)

FIGURE 5-8. The number of visited feature vectors after the second phase filtering (N1) vs. the size of databases for VA-File (a) and the proposed method (b). In both figures, the dashed line is plotted to illustrate a linear increase of N1 as the database grows.